Nakijk 02_03 — V1

NAKIJK V1 · Hoofdstuk 02 · Soft delete en berekeningen · bij Oefening_02_03.md · 17 oktober 2026

Nagekeken: `ProgrammingAdvancedOefeningen` → `Models/Bestellijn.cs` (17 oktober, 07:43) en `Controllers/BestellijnController.cs` (17 oktober, 08:30).

> **OPMERKING**
>
> Nagekeken: punt 1 tot en met 5. De rest maak je nog.
>
> Je vraag: in punt 5 krijg je een melding dat een waarde `null` kan zijn, terwijl je dacht dat je dat al gedekt had. Dat staat bij A1.
>
> De meeste testen van het nakijkscript hebben de POST van punt 6 nodig. In plaats daarvan heb ik een reeks verzoeken voor punt 3 tot 5 nagelopen op je eigen lijst.
>
> Ik kon je API niet starten, want op deze werkruimte staat geen .NET. Ik heb je code gelezen en elk verzoek op papier nagelopen.

In één oogopslag

Punt 1 tot 4 zijn in orde. In punt 5 zit je in de goede richting: je kijkt in één controle of de lijn bestaat én of ze actief is. Alleen het woord dat de twee voorwaarden verbindt, is het verkeerde (A1). Daardoor crasht je API bij een onbekend Id, en geeft hij een geschrapte lijn gewoon terug.

Voor je aan punt 6 begint, zet je best nog één ding recht: je lijst overleeft geen volgend verzoek. Dat is dezelfde fout als in 02_02 (zie **Voor je aan punt 6 begint**).

Punt	Wat de opgave vraagt	Status
1	het model `Bestellijn`	✓ in orde
2	de controller met minstens vijf lijnen, waarvan één geschrapt	✓ in orde, maar zie Voor je aan punt 6 begint
3	`GET api/bestellijn`: enkel de actieve lijnen	✓ in orde
4	`GET api/bestellijn/alles`: alle lijnen	✓ in orde
5	`GET api/bestellijn/{id}`: 200, of 404 als ze niet bestaat of geschrapt is	✗ zie A1, en B1
6 tot 11	toevoegen, bijwerken, schrappen, herstellen, totaal en fiche	nog te doen

Zo antwoordt je API nu, met je eigen vijf lijnen:

Verzoek	Je API antwoordt nu	De opgave vraagt
`GET api/bestellijn`	200 met lijn 1, 2 en 4	✓
`GET api/bestellijn/alles`	200 met de vijf lijnen	✓
`GET api/bestellijn/1`	200 met lijn 1	✓
`GET api/bestellijn/3`, een geschrapte lijn	200 met lijn 3	404 met *We vonden geen actieve bestellijn met Id 3.*
`GET api/bestellijn/99`, een onbekend Id	500, je API crasht	404 met *We vonden geen actieve bestellijn met Id 99.*

A. Wat een verkeerd antwoord geeft

A1. Je controle in punt 5 kijkt naar `IsActief` net wanneer er geen lijn is

Waar: `GetBestellingById`, regel 46.

De melding die je ziet is waarschuwing **CS8602**, Dereference of a possibly null reference, onder `bestelling.IsActief`. De compiler zegt: op deze plaats kan `bestelling` `null` zijn, en toch vraag je er iets van.

Waarom je dacht dat het gedekt was. Er staat `bestelling == null` vóór `bestelling.IsActief`, en dat lijkt een bescherming. Maar of het een bescherming is, hangt af van het woord ertussen.

C# leest een voorwaarde van links naar rechts, en stopt zodra het antwoord vastligt:

Woord	Betekenis	Wanneer kijkt C# naar de rechterkant?
`&&`	en: allebei moeten kloppen	alleen als de linkerkant **klopt**. Klopt links niet, dan is het antwoord al "nee".
`||`	of: één van de twee volstaat	alleen als de linkerkant **niet klopt**. Klopt links wel, dan is het antwoord al "ja".

Met `&&` kijkt C# dus naar `bestelling.IsActief` **precies wanneer** `bestelling == null` **klopt**: wanneer er geen lijn is. Dat is het enige geval waarin het niet kan. De compiler ziet dat, en daarom waarschuwt hij.

Wat er bij elk Id gebeurt:

Verzoek	Links: `bestelling == null`	Rechts	Je API
Id 1, actief	klopt niet, dus het antwoord is "nee"	wordt niet bekeken	200 ✓
Id 3, geschrapt	klopt niet, dus het antwoord is "nee"	wordt niet bekeken	200, maar de opgave wil 404
Id 99, bestaat niet	klopt	wordt bekeken, op een lijn die er niet is	crash: `NullReferenceException`, 500

Twee fouten met één oorzaak: een geschrapte lijn komt erdoor, en een onbekend Id laat je API crashen.

Richting.

- Lees de opgave van punt 5 nog eens: *"Bestaat de lijn niet, of is ze geschrapt, geef dan 404."* De opgave zegt het woord al.
- Kijk in de tabel hierboven welk van de twee woorden de rechterkant alleen bekijkt als er wél een lijn is.
- De volgorde blijft belangrijk: eerst de vraag of de lijn bestaat, dan pas de vraag of ze actief is. Draai je ze om, dan kijk je opnieuw naar `IsActief` voor je weet dat er een lijn is.
- Hoe je twee voorwaarden combineert, staat in je leerboek, hoofdstuk 4. Wat `null` is en waarom de compiler erover waarschuwt, staat in hoofdstuk 17.

Hoe je weet dat het opgelost is. De waarschuwing CS8602 verdwijnt vanzelf. Met het juiste woord weet de compiler dat `bestelling` aan de rechterkant nooit `null` is, en heeft hij niets meer te melden. Doe daarna de drie verzoeken uit de tabel: 200 voor Id 1, en twee keer 404 met het bericht voor Id 3 en Id 99.

> **LET OP**
>
> Zet geen uitroepteken achter `bestelling` om de waarschuwing weg te krijgen. Je leerboek, hoofdstuk 17, legt uit waarom: het uitroepteken is een belofte aan de compiler, geen controle. De waarschuwing verdwijnt, maar je API crasht nog altijd bij Id 99.

B. Werkt, maar wijkt af

B1. Het returntype van punt 5 belooft een lijst

Waar: `GetBestellingById`, regel 42.

Je methode belooft een lijst van bestellijnen, maar geeft er één terug. Het compileert, want `Ok` aanvaardt alles. Maar Scalar toont bij dit endpoint een lijst als antwoord, en dat klopt niet met wat de client krijgt.

Dit was ook punt B1 bij 02_01. Bij je `VentilatorController` deed je het al juist.

Richting. Het returntype zegt wat er terugkomt: hier één bestellijn.

Voor je aan punt 6 begint

Je lijst overleeft geen volgend verzoek

Waar: regel 10.

Je lijst is een gewoon veld van je controller, net als bij je eerste versie van 02_02. Voor punt 3 tot 5 merk je daar niets van, want daar lees je alleen. Vanaf punt 6 wel:

Wat je doet	Wat er gebeurt
een lijn toevoegen (punt 6)	201, maar bij het volgende verzoek is ze weg
een lijn schrappen (punt 8)	204, maar bij het volgende verzoek is ze weer actief, en een tweede schrapping geeft opnieuw 204 in plaats van 404
een lijn herstellen (punt 9)	hetzelfde: het herstel is bij het volgende verzoek vergeten

Richting. Kijk naar regel 7 van je `VentilatorController`. Daar staat het al zoals het hoort, met twee woorden voor `List` die hier ontbreken. Waarom het nodig is, staat in Nakijk_02_02_V2.

De namen: bestelling of bestellijn?

Je lijst heet `bestellingen` en je methodes heten `GetActieveBestellingen` en `GetBestellingById`, terwijl elk item een bestellijn is. Een bestelling bestaat uit meerdere lijnen. In punt 11 schrap je "de hele bestelling", en in hoofdstuk 06 komt er een apart model `Bestelling` bij. Als je dan al `bestellingen` gebruikt voor iets anders, wordt het verwarrend.

Richting. Noem alles wat over één lijn gaat ook zo: `bestellijnen`, en methodes met *Bestellijn* in de naam. Visual Studio kan een naam overal tegelijk aanpassen met **F2** op de naam.

C. Opkuis, zonder invloed op het antwoord

Waar	Wat	Richting
regel 1 en 2	De twee `using`-regels staan ook al in `GlobalUsings.cs`. Visual Studio toont ze grijs.	Weghalen; ze doen niets.
`Bestellijn.cs`, regel 9	`= false` achter `IsActief`. Een `bool` begint al op `false`, dus dit doet niets. In punt 6 moet een nieuwe lijn bovendien altijd actief zijn, en dat zet je daar zelf.	Mag weg.
regel 26	`== true` achter `bestelling.IsActief`. `IsActief` is zelf al waar of onwaar.	Zonder `== true` leest het als een zin: *als de bestelling actief is*.

D. Wat goed is, en waarom

- **Het model.** Vijf eigenschappen met het juiste type: een `double` voor de stukprijs, een kommagetal, en een `bool` voor *ja of nee*.
- **Je testdata.** Vijf lijnen, waarvan twee al geschrapt. De opgave vraagt er minstens één. Met twee kan je straks beter zien of je totaal en je fiche de geschrapte lijnen echt overslaan.
- **Punt 3 filtert zonder iets te veranderen.** Je maakt een nieuwe lijst met de actieve lijnen, en je oorspronkelijke lijst blijft volledig. Dat is precies wat een soft delete vraagt: niets verdwijnt, de client ziet het alleen niet.
- **Punt 4 en de route `alles`.** `GET api/bestellijn/alles` komt bij `GetAlleBestellingen` uit en niet bij `GetBestellingById`, ook al past *alles* op de plaats van `{id}`. Een vast woord in een route weegt zwaarder dan een parameter. Dat komt van pas bij `totaal` in punt 10 en `fiche` in punt 11.
- `Bestellijn?` **op regel 44.** Het vraagteken klopt, want `FirstOrDefault` kan niets vinden. Daardoor kan de compiler je nu ook waarschuwen bij A1.
- **Het idee achter punt 5.** Twee redenen voor dezelfde 404 in één controle, met de vraag naar het bestaan vooraan. Alleen het woord ertussen moet anders.
- **Het bericht van punt 5** is letterlijk dat van de opgave.

Zo kan het ook. Je lus in punt 3 werkt goed. Je leerboek, hoofdstuk 11, toont in de tabel *Lussen die LINQ voor je schrijft* hoe dezelfde filtering in één regel kan. Dat hoeft niet, het is een andere manier om hetzelfde te schrijven.

Compilerwaarschuwingen

Voor zover ik kan nagaan door te lezen, geeft je code **één** waarschuwing: **CS8602** op regel 46, bij A1. Ze verdwijnt zodra de controle het juiste woord gebruikt.

In Visual Studio zie je ze in de Error List, bij **Warnings**.

Zelf nakijken

Verzoek	Wat je moet zien	Hoort bij
bouwen met Ctrl+Shift+B	geen waarschuwing CS8602 meer	A1
`GET api/bestellijn/1`	200 met lijn 1	punt 5
`GET api/bestellijn/3`	404 met *We vonden geen actieve bestellijn met Id 3.*	A1
`GET api/bestellijn/99`	404 met *We vonden geen actieve bestellijn met Id 99.*	A1
`GET api/bestellijn` en `GET api/bestellijn/alles`	drie en vijf lijnen	punt 3 en 4

Het nakijkscript kan je nu al draaien, maar de meeste testen voegen eerst een lijn toe met de POST van punt 6. Zolang die er niet is, falen ze allemaal, ook die over punt 5. Alleen de eerste twee testen, voor punt 3 en punt 4, zeggen nu al iets.

Vragen die je je misschien stelt

Waarom crasht mijn API bij Id 99, en zegt de compiler alleen "waarschuwing"? Een waarschuwing laat je programma toch bouwen. De compiler zegt: dit kan misgaan. Bij Id 99 gaat het ook echt mis. Draai je je API met F5, dan stopt Visual Studio op regel 46 met een `NullReferenceException`. Zonder debugger krijgt de client een 500.

Kan ik de twee voorwaarden niet in twee aparte controles zetten? Dat kan ook, en het geeft hetzelfde antwoord: eerst de vraag of de lijn bestaat, dan de vraag of ze actief is, telkens met dezelfde 404 en hetzelfde bericht. Het is langer, maar je ziet elke reden apart. Kies wat je zelf het vlotst leest.

Waarom moet een geschrapte lijn een 404 geven? Ze bestaat toch nog? In je lijst wel, maar voor deze client niet. Dat is wat een soft delete betekent: de gegevens blijven bewaard voor de boekhouding, en die ziet ze via `alles`. Voor wie één lijn opvraagt, is een geschrapte lijn verdwenen.